

EC4010
DIGITAL DESIGN
ANALYSIS AND DESIGN OF
ASYNCHRONOUS SEQUENTIAL
CIRCUITS (PART 2)

BANDARA H.G.T.D.

2022/E/048

GROUP CG04

SEMESTER 04

19 JAN 2025

TITLE: ANALYSIS AND DESIGN OF ASYNCHRONOUS SEQUENTIAL CIRCUITS (PART 2)

OBJECTIVES:

Structural modeling asynchronous sequential circuits.

Determining outputs of an asynchronous sequential circuit for a given sequence of inputs.

Identify stability considerations.

Identify and mitigate race conditions.

TASK 01: Modeling of an asynchronous sequential circuit

(a)

TABLE 1: TRANSITION TABLE

Present State		Inputs		Next State	
$y_{1(t)}$	$y_{2(t)}$	x_1	x_2	$y_{1(t+1)}$	$y_{2(t+1)}$
0	0	0	0	0	0
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	0	1
0	1	1	0	1	1
0	1	1	1	0	1
1	0	0	0	0	0
1	0	0	1	1	1
1	0	1	0	0	0
1	0	1	1	1	1
1	1	0	0	0	0
1	1	0	1	1	1
1	1	1	0	0	1
1	1	1	1	1	1

(b)

```
question_01.v X
1 `timescale 1 ps/1 ps
2 module Question_01(x1,x2,Y1,Y2);
3
4   input x1,x2;
5   output Y1,Y2;
6   wire w1,w2,w3,w4,w5;
7
8   not #10 G1 (w1,x1);
9   not #10 G2 (w5,x2);
10  nand #10 G3 (w2,w1,x2);
11  nand #10 G4 (w3,x2,Y1);
12  nand #10 G5 (Y1,w2,w3);
13  nand #10 G6 (w4,x1,Y2);
14  nand #10 G7 (Y2,w4,w5);
15
16 endmodule
17
18
```

FIGURE 01: Verilog Code

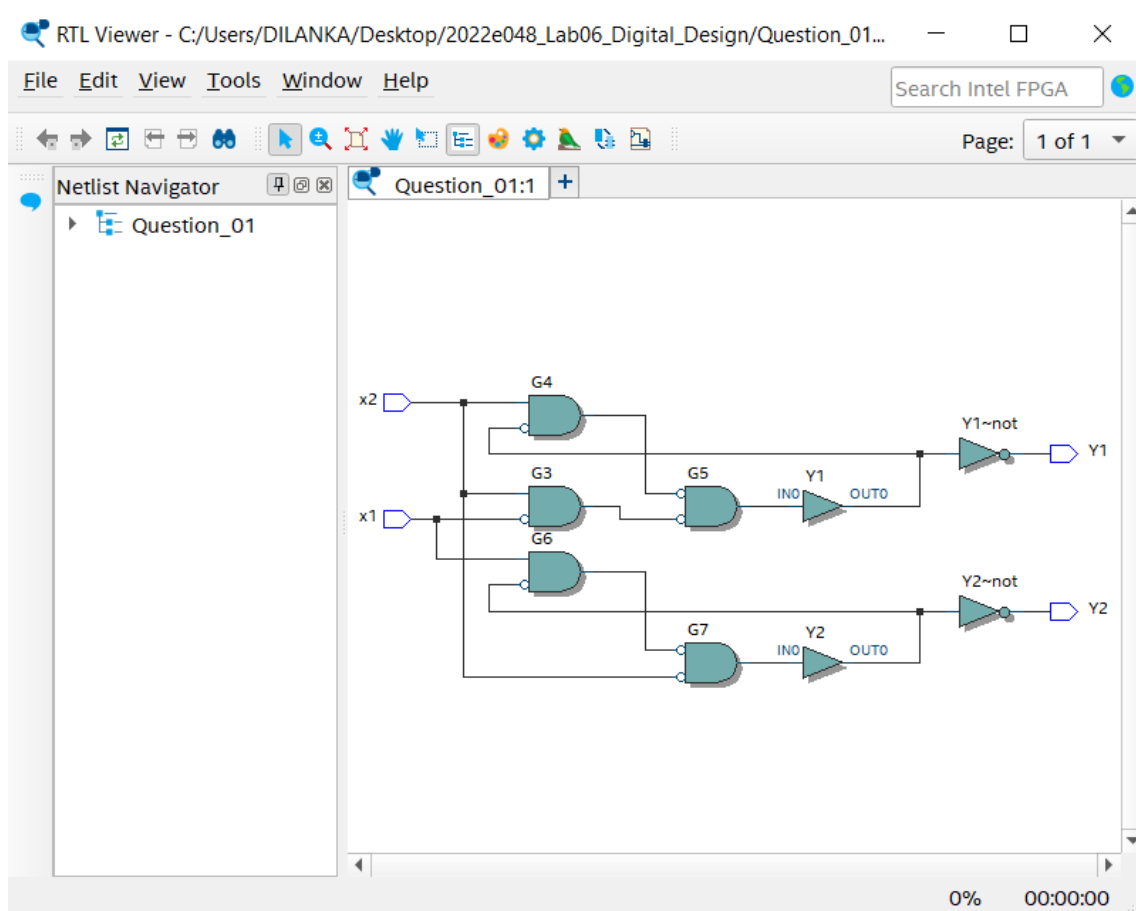


FIGURE 02: Obtained Circuit using RTL Viewer

(c)

```
question_01.v X Question_01_testbench.v X
1 | timescale 1 ps/1 ps
2
3 module Question_01_testbench;
4
5     reg x1,x2;
6     wire y1,y2;
7
8     Question_01 u1(
9
10         .x1(x1),
11         .x2(x2),
12         .y1(y1),
13         .y2(y2)
14     );
15
16
17     initial
18     begin
19         x1=0; x2=0;
20         #10 x1=1; x2=0;
21         #10 x1=1; x2=1;
22         #10 x1=0; x2=1;
23         #10 x1=1; x2=1;
24         #10 x1=1; x2=0;
25         #10 x1=0; x2=0;
26         #20
27         $stop;
28         $finish;
29     end
```

FIGURE 03: Testbench

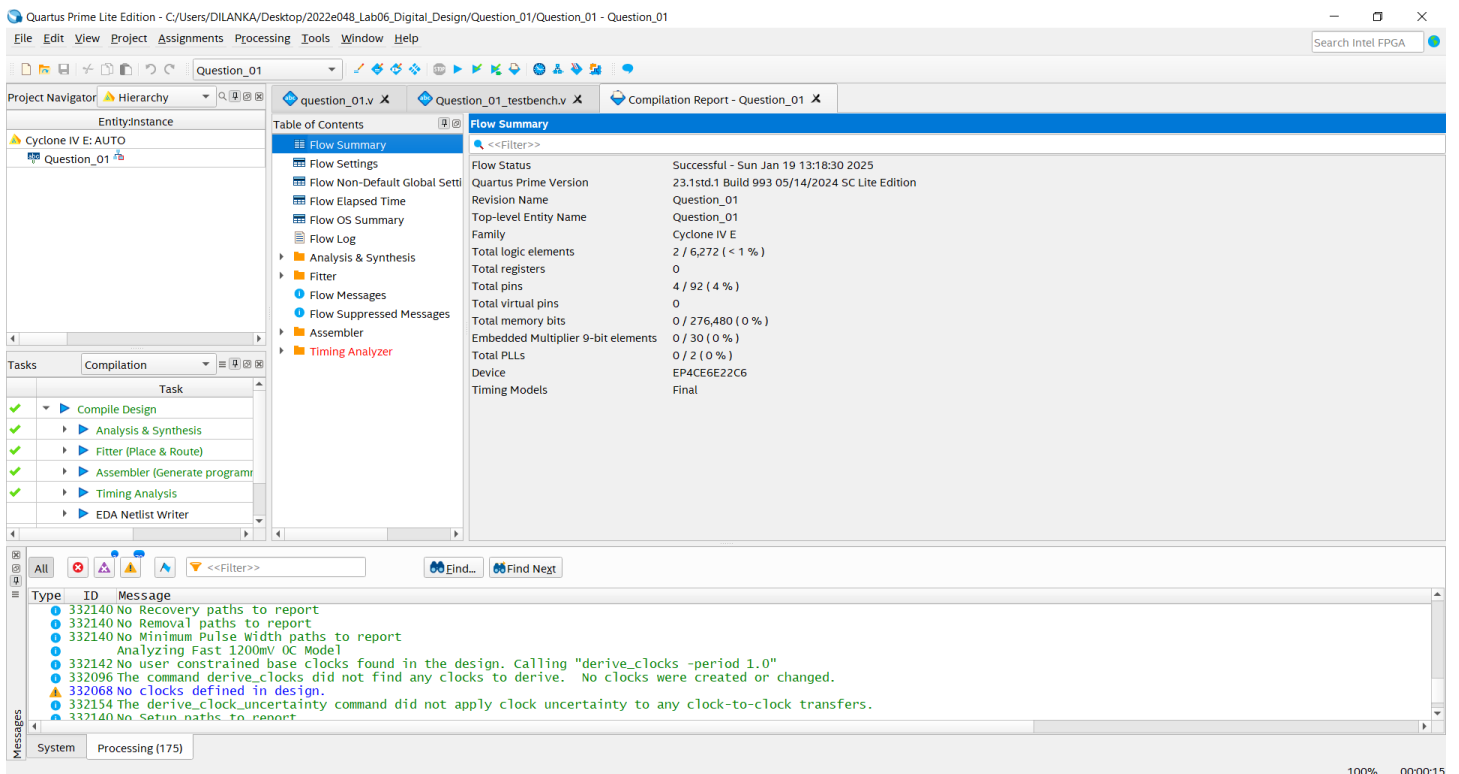


FIGURE 04: Compilation Report

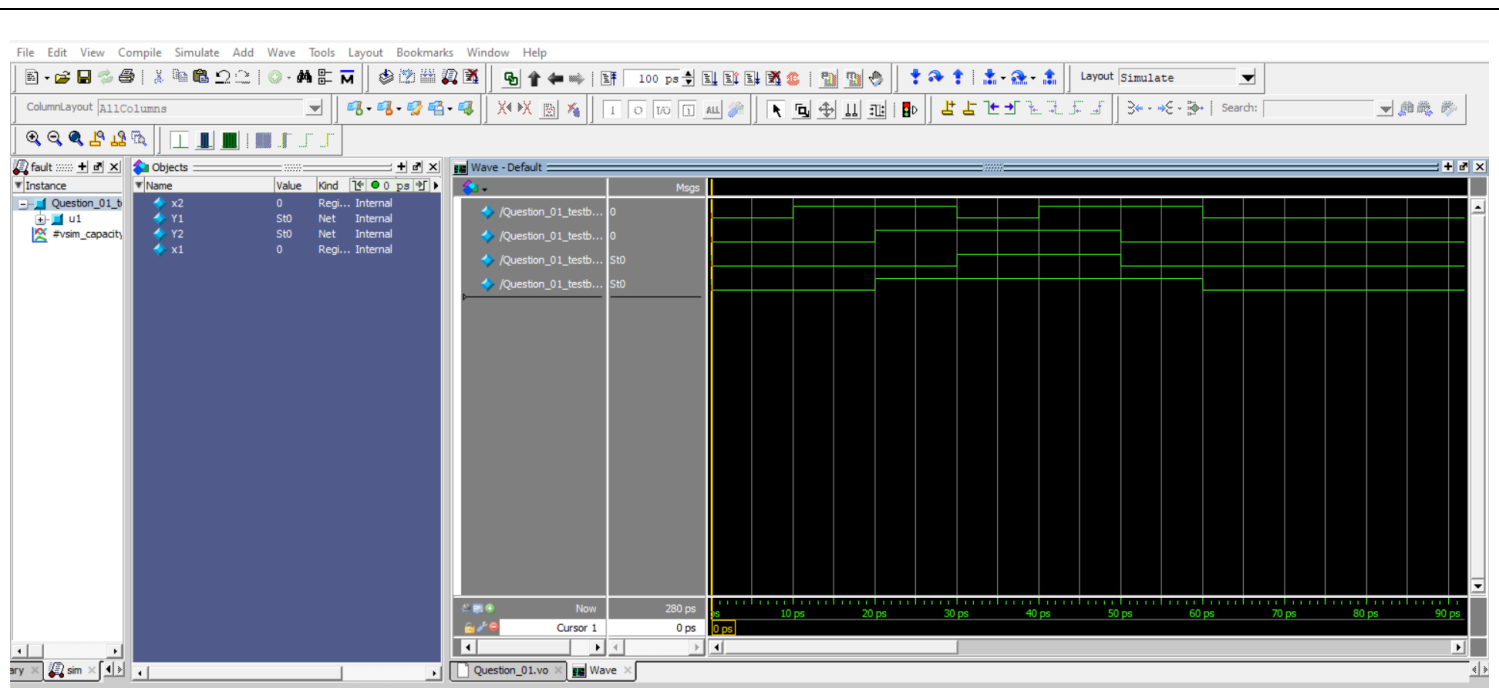


FIGURE 05: Waveform

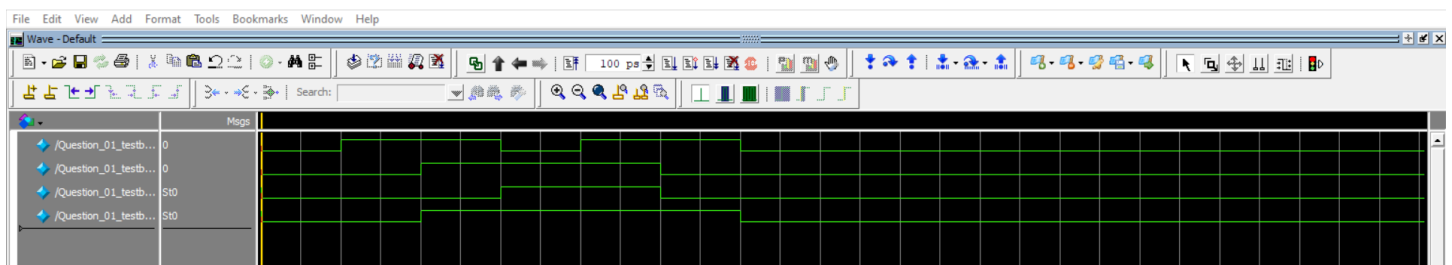


FIGURE 06: Output Waveform

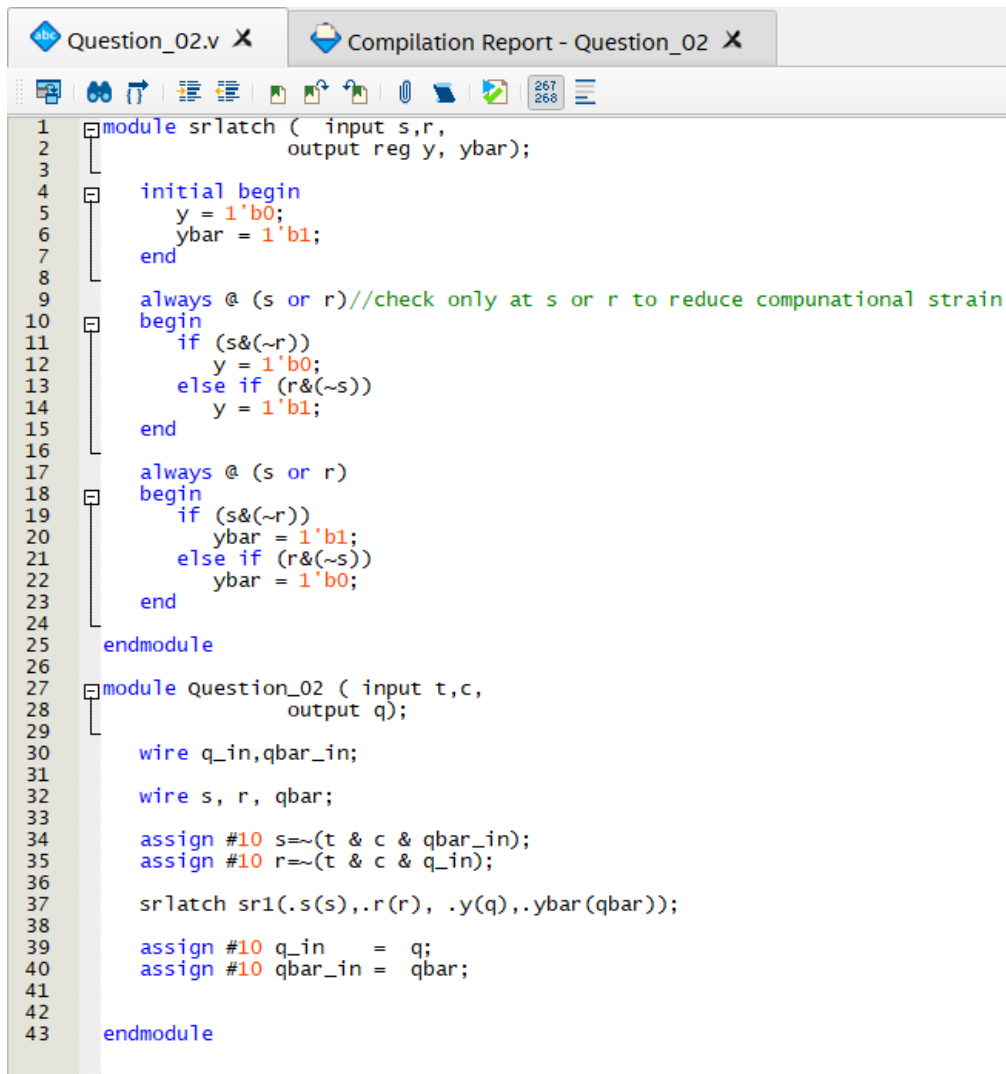
TASK 02:

(a) $S = \overline{TCQ'}$ $R = \overline{TCQ}$

(b) **TABLE 2: TRANSITION TABLE**

T	C	Q	Q'	S	R	Q* (Next State)
0	0	0	1	1	1	0
0	0	1	0	1	1	0
0	1	0	1	1	1	0
0	1	1	0	1	1	1
1	0	0	1	1	1	1
1	0	1	0	1	1	1
1	1	0	1	0	1	1
1	1	1	0	1	0	0

(c)



```

1 module srlatch ( input s,r,
2                 output reg y, ybar);
3
4     initial begin
5         y = 1'b0;
6         ybar = 1'b1;
7     end
8
9     always @ (s or r)//check only at s or r to reduce compunational strain
10    begin
11        if (s&(~r))
12            y = 1'b0;
13        else if (r&(~s))
14            y = 1'b1;
15    end
16
17    always @ (s or r)
18    begin
19        if (s&(~r))
20            ybar = 1'b1;
21        else if (r&(~s))
22            ybar = 1'b0;
23    end
24 endmodule
25
26 module Question_02 ( input t,c,
27                     output q);
28
29     wire q_in,qbar_in;
30
31     wire s, r, qbar;
32
33     assign #10 s=~(t & c & qbar_in);
34     assign #10 r=~(t & c & q_in);
35
36     srlatch sr1(.s(s),.r(r), .y(q),.ybar(qbar));
37
38     assign #10 q_in = q;
39     assign #10 qbar_in = qbar;
40
41
42 endmodule
43
  
```

FIGURE 07: Verilog Code

```

1  module flip_test();
2
3      reg t_in, c_in;
4      wire q_out;
5
6      flip flip_dut(.t(t_in), .c(c_in), .q(q_out));
7
8      // t a shifted square wave of period 80000 (shisted by pi/8)
9      always
10     begin
11         t_in = 1'b1;
12         t_in = #10000 1'b0;
13         t_in = #40000 1'b1;
14         #30000;
15     end
16
17     // c a square pulse of period 40000
18     always
19     begin
20         repeat(4)
21         begin
22             c_in = 1'b0;
23             c_in = #20000 1'b1;
24             #20000;
25         end
26         repeat(4)
27         begin
28             c_in = 1'b1;
29             c_in = #20000 1'b0;
30             #20000;
31         end
32     end
33
34 endmodule
35

```

FIGURE 08: Testbench

Quartus Prime Lite Edition - C:/Users/DILANKA/Desktop/2022e048_Lab06_Digital_Design/Question_02/Question_02 - Question_02

File Edit View Project Assignments Processing Tools Window Help

Search Intel FPGA

Project Navigator Hierarchy

Entity/Instance

Cyclone IV E: AUTO

Question_02

Table of Contents

Flow Summary

Flow Settings

Flow Non-Default Global Settings

Flow Elapsed Time

Flow OS Summary

Flow Log

Analysis & Synthesis

Fitter

Flow Messages

Flow Suppressed Messages

Assembler

Timing Analyzer

Flow Status

Quartus Prime Version

Revision Name

Top-level Entity Name

Family

Total logic elements

Total registers

Total pins

Total virtual pins

Total memory bits

Embedded Multiplier 9-bit elements

Total PLLs

Device

Timing Models

Successful - Sun Jan 19 13:45:33 2025

23.1std.1 Build 993 05/14/2024 SC Lite Edition

Question_02

Question_02

Cyclone IV E

4 / 6,272 (< 1 %)

0

3 / 92 (3 %)

0

0 / 276,480 (0 %)

0 / 30 (0 %)

0 / 2 (0 %)

EP4CE6E22C6

Final

Tasks

Compilation

Task

Compile Design

Analysis & Synthesis

Fitter (Place & Route)

Assembler (Generate program)

Timing Analysis

EDA Netlist Writer

Edit Settings

Program Device (Open Programme)

Messages

System (2)

Processing (180)

Quartus Prime Timing Analyzer was successful. 0 errors, 18 warnings

293000 Quartus Prime Full Compilation was successful. 0 errors, 75 warnings

100% 00:00:16

FIGURE 09: Compilation Report

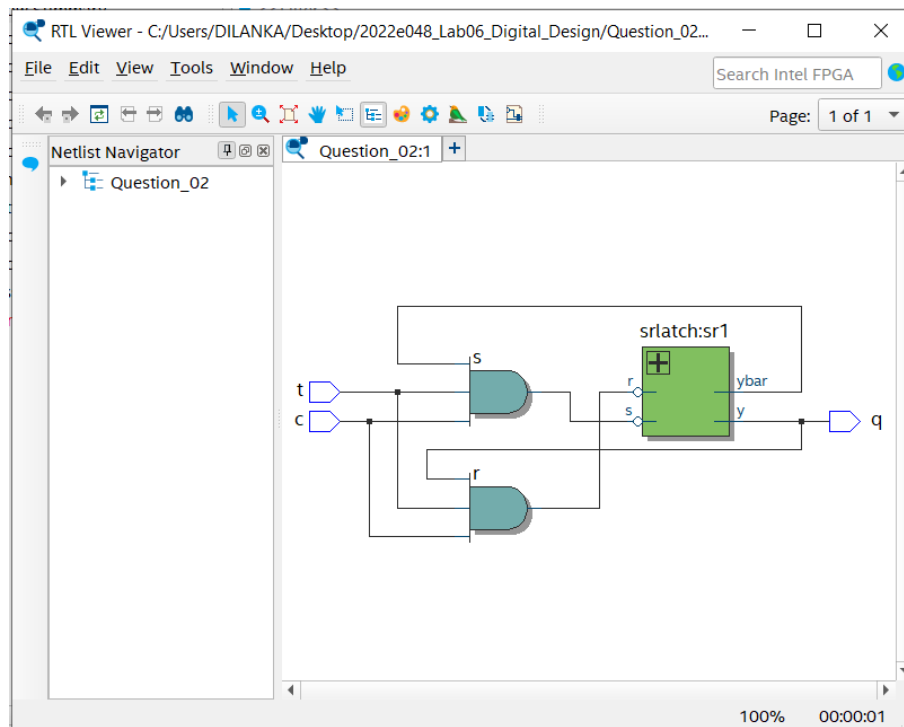


FIGURE 10: Obtained Circuit using RTL Viewer

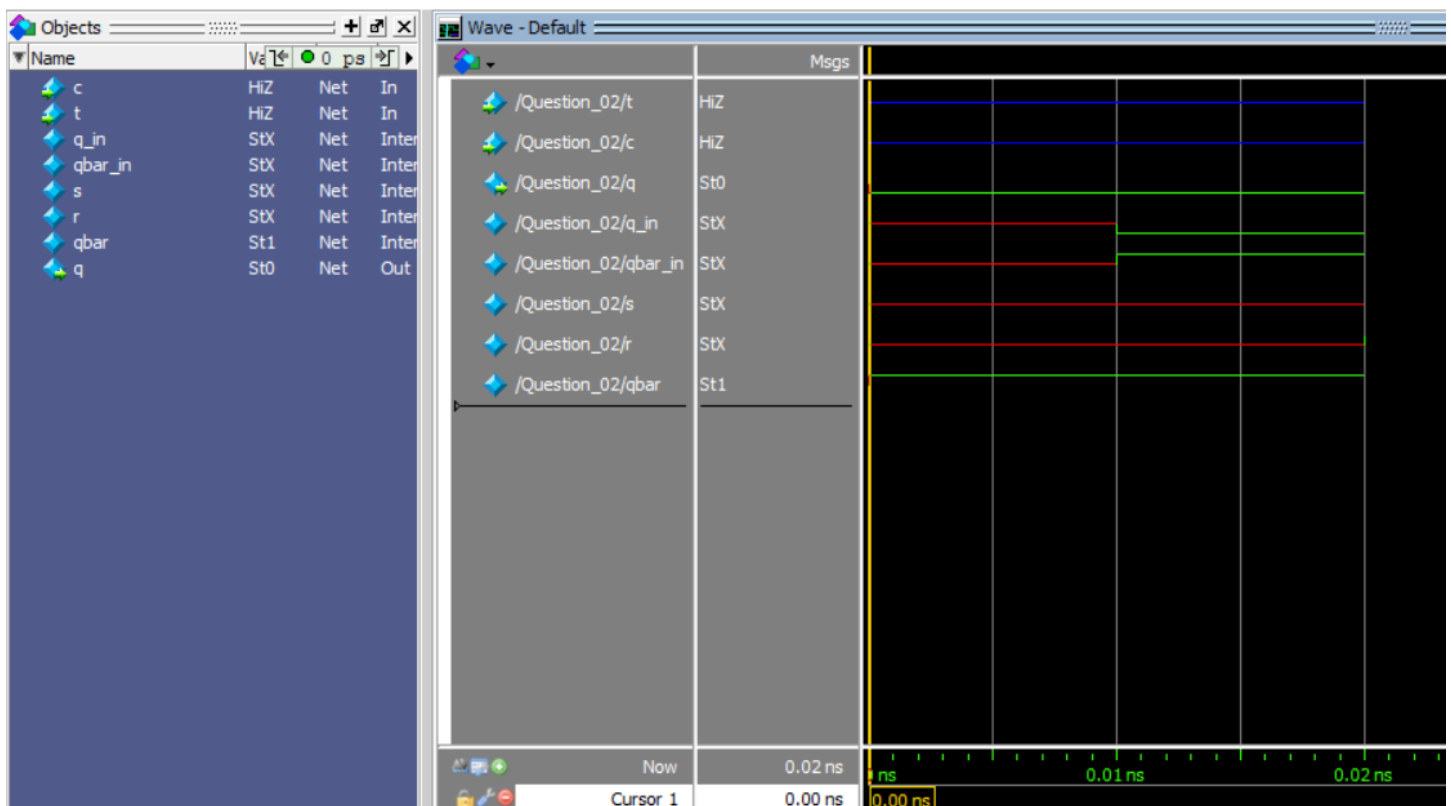
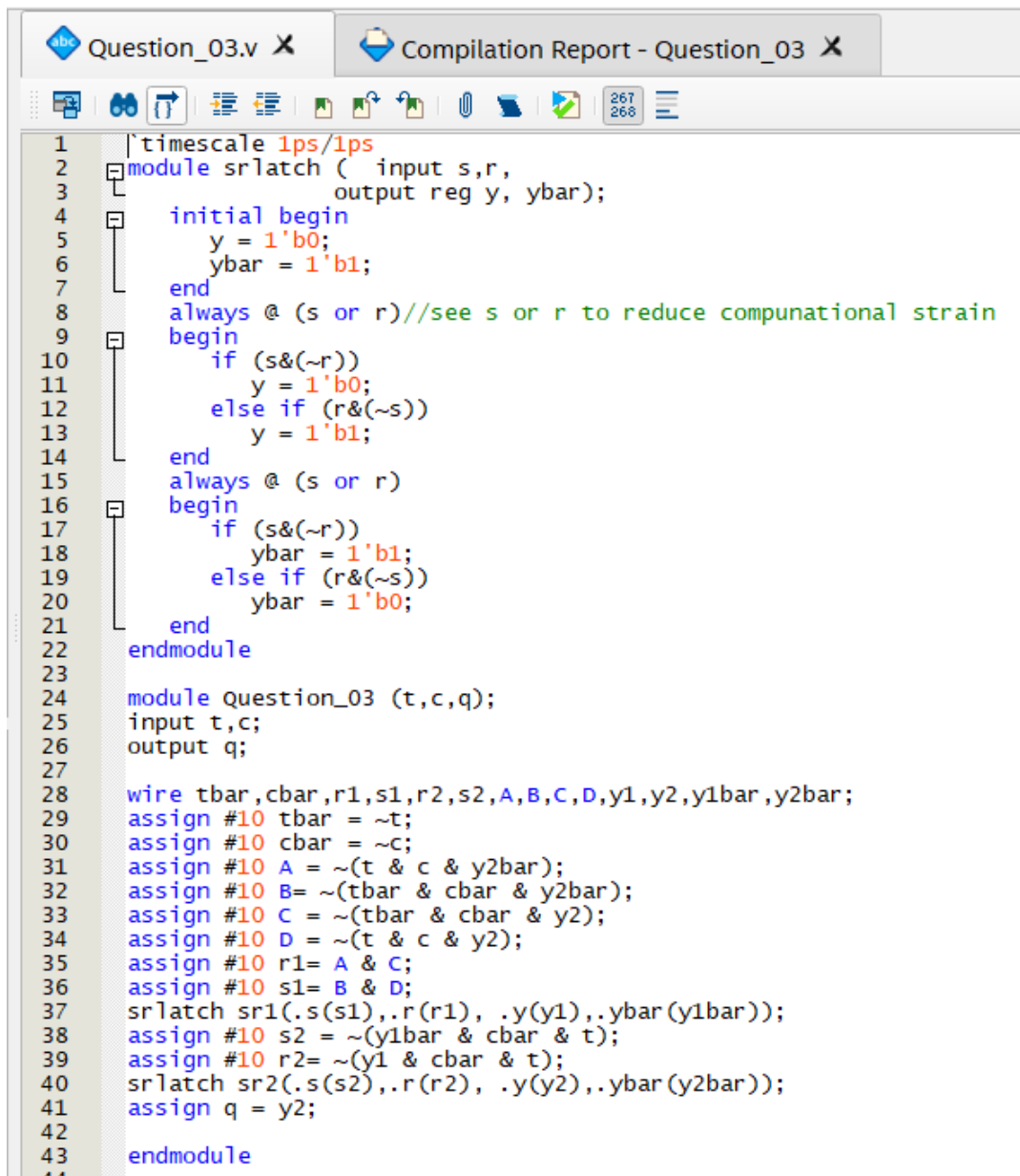


FIGURE 11: Output Waveform

TASK 03:

(a)



```
1 |`timescale 1ps/1ps
2 |module srlatch ( input s,r,
3 |                output reg y, ybar);
4 |    initial begin
5 |        y = 1'b0;
6 |        ybar = 1'b1;
7 |    end
8 |    always @ (s or r)//see s or r to reduce compunational strain
9 |    begin
10 |        if (s&(~r))
11 |            y = 1'b0;
12 |        else if (r&(~s))
13 |            y = 1'b1;
14 |    end
15 |    always @ (s or r)
16 |    begin
17 |        if (s&(~r))
18 |            ybar = 1'b1;
19 |        else if (r&(~s))
20 |            ybar = 1'b0;
21 |    end
22 |endmodule
23
24 |module Question_03 (t,c,q);
25 |input t,c;
26 |output q;
27
28 |wire tbar,cbar,r1,s1,r2,s2,A,B,C,D,y1,y2,y1bar,y2bar;
29 |assign #10 tbar = ~t;
30 |assign #10 cbar = ~c;
31 |assign #10 A = ~(t & c & y2bar);
32 |assign #10 B= ~(tbar & cbar & y2bar);
33 |assign #10 C = ~(tbar & cbar & y2);
34 |assign #10 D = ~(t & c & y2);
35 |assign #10 r1= A & C;
36 |assign #10 s1= B & D;
37 |srlatch sr1(.s(s1),.r(r1), .y(y1),.ybar(y1bar));
38 |assign #10 s2 = ~(y1bar & cbar & t);
39 |assign #10 r2= ~(y1 & cbar & t);
40 |srlatch sr2(.s(s2),.r(r2), .y(y2),.ybar(y2bar));
41 |assign q = y2;
42
43 |endmodule
44
```

FIGURE 12: Verilog Code

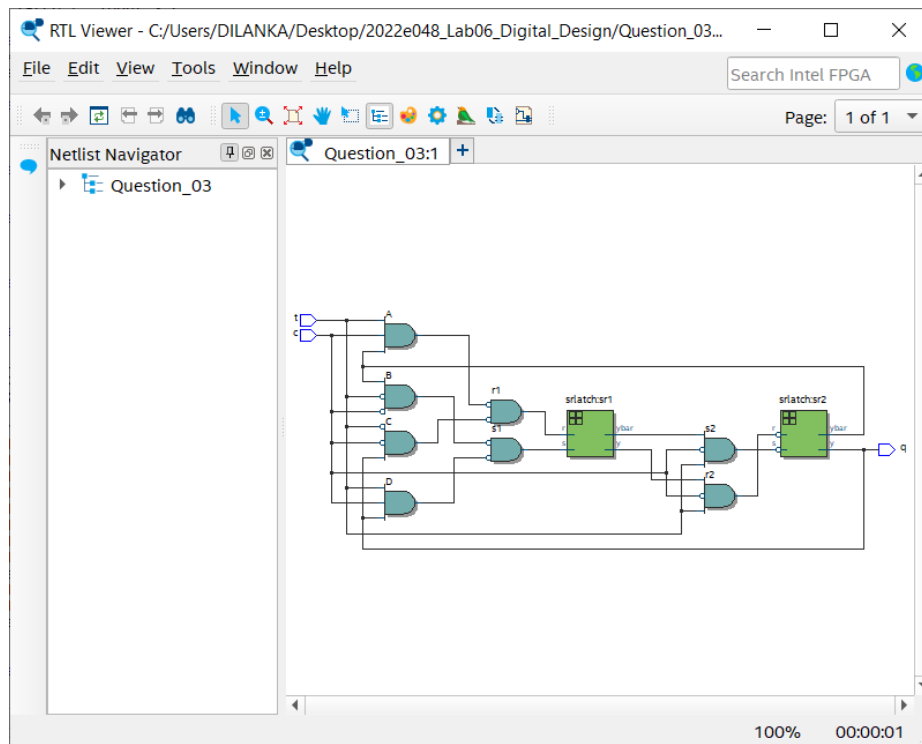


FIGURE 13: Obtained Circuit using RTL Viewer

(b)

```

1  module flip_test();
2
3      reg t_in, c_in;
4      wire q_out;
5
6      flip flip_dut(.t(t_in), .c(c_in), .q(q_out));
7
8      // t a shifted square wave of period 80000 (shisted by pi/8)
9      always
10     begin
11         t_in = 1'b1;
12         t_in = #10000 1'b0;
13         t_in = #40000 1'b1;
14         #30000;
15     end
16
17     // c a square pulse of period 40000
18     always
19     begin
20         repeat(4)
21         begin
22             c_in = 1'b0;
23             c_in = #20000 1'b1;
24             #20000;
25         end
26         repeat(4)
27         begin
28             c_in = 1'b1;
29             c_in = #20000 1'b0;
30             #20000;
31         end
32     end
33 endmodule
34
35

```

FIGURE 14: Testbench

c)

A negative edge-triggered T flip-flop is an electronic circuit designed to store and toggle a single bit of information, either 0 or 1. It operates by responding to a voltage change on its "clock" input, which triggers the output to switch between its current state and the opposite state.

Gate-level simulation is a technique used to verify the functionality of digital circuits by emulating the behavior of individual logic gates within the circuit. For a negative edge-triggered T flip-flop, this involves simulating the operation of components like AND gates, NOR gates, and inverters that form the flip-flop.

The simulation models the inputs and outputs of each gate and examines their interactions according to the circuit's logic. When a negative edge is detected on the clock input, the simulation replicates the flip-flop's behavior, toggling the output based on the T input value.

By simulating the individual gates comprising the flip-flop, gate-level simulation ensures the circuit functions correctly and meets the intended design specifications.